In the delimiter checker, every stack element holds an opening bracket that remains unclosed. For instance, in ([{}]), the application adds the opening brackets to the stack. When it encounters }, it must pop { first since that was the most recently added bracket. This is important because the brackets need to close in the reverse order of how they were opened. Removing the oldest bracket first would lead to an incorrect matching by the program.

In the postfix calculator, every element in the stack holds a number or a calculation result. For instance, in 10 4 -, the program places 10 and then 4 onto the stack. When it hits -, it first releases 4, followed by 10. It computes 10 - 4 = 6 and places 6 back onto the stack. The sequence matters because if we compute 4 - 10, we will obtain -6, which is incorrect.

Both programs utilize LIFO (Last In, First Out), which indicates that the most recently added item is the first to be taken out. The delimiter checker utilizes this to align brackets in the proper sequence, while the postfix calculator employs it to verify that the computations are accurate.